

LAB EXAM MODEL QUESTIONS

5. Develop a Python GUI-based application to manage a basic hostel room allocation system. The application should allow the user to add room details and student information. Design classes to represent rooms and allocation operations. The system should support room allocation and deallocation, ensuring that a room cannot be assigned if it is already occupied. Provide a feature to search for room availability and display lists of occupied and vacant rooms. Maintain a log file for all allocation and deallocation activities.

ALGORITHM: HOSTEL ROOM ALLOCATION SYSTEM

1. Start
2. Import required modules (tkinter)
3. Define a class Room with attributes:
 - Room number
 - Occupied status (True/False)
 - Student name
4. Define methods in class:
 - allocate(student_name) →
 - If room vacant → assign student
 - Else → show “Room already occupied”
 - deallocate() →
 - Remove student and mark room vacant
 - is_vacant() → return True if room is free
5. Create a dictionary to store rooms
6. Define function to log allocation/deallocation into file
7. Create GUI window using Tkinter
8. Add input fields:

- Room number
- Student name

9. Add buttons:

- Add Room
- Allocate Room
- Deallocate Room
- Search Room
- Show Status

10. On "Add Room":

- Create Room object
- Store in dictionary

11. On "Allocate Room":

- Check if room exists and vacant
- Allocate to student
- Log action

12. On "Deallocate Room":

- Free the room
- Log action

13. On "Search Room":

- Display room status

14. On "Show Status":

- Display occupied and vacant rooms

15. Run GUI loop (mainloop())

16. Stop

```

# =====
# PROGRAM: HOSTEL ROOM ALLOCATION SYSTEM (GUI + OOP)
# =====

import tkinter as tk
from tkinter import messagebox

# =====
# STEP 1: ROOM CLASS
# =====

class Room:
    def __init__(self, room_no):
        self.room_no = room_no
        self.occupied = False
        self.student = ""

    def allocate(self, student_name):
        if self.occupied:
            return "Room already occupied"
        else:
            self.student = student_name
            self.occupied = True
            log_action(f"Room {self.room_no} allocated to {student_name}")
            return "Room allocated"

    def deallocate(self):

```

```

    if self.occupied:
        log_action(f"Room {self.room_no} deallocated from {self.student}")
        self.student = ""
        self.occupied = False
        return "Room deallocated"
    else:
        return "Room already vacant"

```

```

def is_vacant(self):
    return not self.occupied

```

```

# =====

```

```

# STEP 2: STORAGE

```

```

# =====

```

```

rooms = {}

```

```

# =====

```

```

# STEP 3: LOG FUNCTION

```

```

# =====

```

```

def log_action(text):
    with open("room_log.txt", "a") as f:
        f.write(text + "\n")

```

```

# =====

```

STEP 4: GUI FUNCTIONS

=====

```
def add_room():
```

```
    room_no = room_entry.get()
```

```
    if room_no not in rooms:
```

```
        rooms[room_no] = Room(room_no)
```

```
        messagebox.showinfo("Success", "Room added")
```

```
    else:
```

```
        messagebox.showerror("Error", "Room already exists")
```

```
def allocate_room():
```

```
    room_no = room_entry.get()
```

```
    student = student_entry.get()
```

```
    if room_no in rooms:
```

```
        msg = rooms[room_no].allocate(student)
```

```
        messagebox.showinfo("Allocate", msg)
```

```
    else:
```

```
        messagebox.showerror("Error", "Room not found")
```

```
def deallocate_room():
```

```
    room_no = room_entry.get()
```

```
    if room_no in rooms:
```

```
msg = rooms[room_no].deallocate()
messagebox.showinfo("Deallocate", msg)
else:
    messagebox.showerror("Error", "Room not found")
```

```
def search_room():
    room_no = room_entry.get()

    if room_no in rooms:
        r = rooms[room_no]
        output.delete("1.0", tk.END)

        if r.occupied:
            output.insert(tk.END, f"Room {room_no} occupied by {r.student}")
        else:
            output.insert(tk.END, f"Room {room_no} is vacant")
    else:
        messagebox.showerror("Error", "Room not found")
```

```
def show_status():
    output.delete("1.0", tk.END)

    output.insert(tk.END, "Occupied Rooms:\n")
    for r in rooms.values():
```

```

        if not r.is_vacant():
            output.insert(tk.END, f"{r.room_no} - {r.student}\n")

output.insert(tk.END, "\nVacant Rooms:\n")
for r in rooms.values():
    if r.is_vacant():
        output.insert(tk.END, f"{r.room_no}\n")

# =====
# STEP 5: GUI DESIGN
# =====

root = tk.Tk()
root.title("Hostel Room System")

tk.Label(root, text="Room Number").pack()
room_entry = tk.Entry(root)
room_entry.pack()

tk.Label(root, text="Student Name").pack()
student_entry = tk.Entry(root)
student_entry.pack()

tk.Button(root, text="Add Room", command=add_room).pack()
tk.Button(root, text="Allocate Room", command=allocate_room).pack()
tk.Button(root, text="Deallocate Room", command=deallocate_room).pack()

```

```
tk.Button(root, text="Search Room", command=search_room).pack()
```

```
tk.Button(root, text="Show Status", command=show_status).pack()
```

```
output = tk.Text(root, height=10)
```

```
output.pack()
```

```
root.mainloop()
```